FairnessOps: Complete Tech Stack & ML/DL Techniques

Part 1: Core Tech Stack

Language: Python

Why:

- ML/DL standard
- Libraries (PyTorch, scikit-learn, SHAP)
- Easy to learn, fast to code
- Hospital data scientists know it

Version: 3.9+

Setup:

```bash
python3 --version  # Check version
python3 -m venv venv  # Create environment
source venv/bin/activate  # Activate
```

Part 2: Data Processing & Computation

NumPy

What: Array operations, math **Why:** Foundation for everything else **When to use:** Computing metrics, bootstrap, statistics

Installation:

```bash
pip install numpy==1.24.0
```

Key functions:

```python
```

```python
import numpy as np

# Create arrays
x = np.array([1, 2, 3])
predictions = np.random.uniform(0, 1, 1000)

# Operations
mean = np.mean(predictions)
std = np.std(predictions)
percentile = np.percentile(predictions, 95)

# Indexing/masking
mask = demographics == 'Black'
subset = predictions[mask]

# Bootstrap resampling
for i in range(500):
    idx = np.random.choice(len(data), len(data), replace=True)
    resampled = data[idx]
```

Pandas

What: Data manipulation, dataframes **Why:** Load, clean, organize data **When to use:** Working with tabular data (patients, features, outcomes)

Installation:

```bash
pip install pandas==2.0.0
```

Key functions:

```python
```

```python
import pandas as pd

# Load data
df = pd.read_csv('MIMIC_sepsis.csv')

# Inspect
print(df.head())
print(df.describe())
print(df.isna().sum())

# Filter/groupby
white_patients = df[df['race'] == 'White']
by_race = df.groupby('race')['auc'].mean()

# Create new columns
df['risk_category'] = pd.cut(df['prediction'], bins=[0, 0.3, 0.7, 1.0])
```

Part 3: ML Metrics & Statistics

Scikit-Learn

What: Machine learning metrics, utilities **Why:** Industry standard for ML evaluation **When to use:** Computing AUC, confusion matrix, calibration, train/test split

Installation:

```bash
pip install scikit-learn==1.2.0
```

Key functions:

Classification Metrics

```python
```

```python
from sklearn.metrics import (
    roc_auc_score,
    confusion_matrix,
    calibration_curve,
    roc_curve
)

# AUC
auc = roc_auc_score(y_true, y_pred)

# Confusion matrix
tn, fp, fn, tp = confusion_matrix(y_true, y_pred_binary).ravel()
sensitivity = tp / (tp + fn)
specificity = tn / (tn + fp)

# Calibration
prob_true, prob_pred = calibration_curve(y_true, y_pred, n_bins=10)
calibration_error = np.mean(np.abs(prob_true - prob_pred))
```

Train/Test Split

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,  # Keep class balance
    random_state=42
)
```

Preprocessing

```python
```

```python
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

# Encode categorical variables
encoder = LabelEncoder()
race_encoded = encoder.fit_transform(['Black', 'White', 'Asian'])
```

SciPy

What: Statistical tests, advanced math **Why:** Statistical rigor (p-values, KS tests, etc.) **When to use:** Hypothesis testing, distribution comparisons, significance tests

Installation:

```bash
pip install scipy==1.10.0
```

Key functions:

Statistical Tests

```python
from scipy import stats

# KS test (feature drift)
ks_stat, p_value = stats.ks_2samp(baseline_feature, current_feature)

# T-test
t_stat, p_value = stats.ttest_ind(group1, group2)

# Chi-square
chi2, p_value = stats.chisquare(observed, expected)
```

Distributions

```python
```

```python
# Generate from distribution
normal_data = stats.norm.rvs(loc=0, scale=1, size=1000)

# CDF/PDF
cdf_value = stats.norm.cdf(2.0)
pdf_value = stats.norm.pdf(2.0)
```

Part 4: Deep Learning (Neural Networks)

PyTorch

What: Neural network framework **Why:** Industry standard, flexible, easy to debug **When to use:** Training adversary network, any deep learning

Installation:

```bash
pip install torch==2.0.0
```

Key components:

Tensors (PyTorch arrays)

```python
import torch

# Create tensors
x = torch.tensor([1.0, 2.0, 3.0])
x = torch.zeros(5, 3)
x = torch.randn(1000, 10)  # Random normal

# Operations
y = x * 2
z = torch.mean(x)

# Convert to/from numpy
numpy_array = x.numpy()
torch_tensor = torch.from_numpy(numpy_array)
```

Neural Network Basics

```python
import torch.nn as nn

# Define network
class SimpleNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(1, 16)    # Input layer
        self.relu = nn.ReLU()              # Activation
        self.linear2 = nn.Linear(16, 2)    # Output layer

    def forward(self, x):
        x = self.linear1(x)
        x = self.relu(x)
        x = self.linear2(x)
        return x

# Create network
model = SimpleNetwork()
```

Training Loop

```python
```

```python
import torch.optim as optim

# Setup
optimizer = optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()

# Training
for epoch in range(300):
    # Forward pass
    predictions = model(X)

    # Compute loss
    loss = loss_fn(predictions, y)

    # Backward pass
    optimizer.zero_grad()  # Clear gradients
    loss.backward()        # Compute gradients
    optimizer.step()       # Update weights

# Evaluation
model.eval()  # Switch to evaluation mode
with torch.no_grad():
    final_predictions = model(X_test)
```

Adversary Network (Your Main Tool)

```python
```

```python
class AdversaryNetwork(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 32),           # Input: 1 (model prediction)
            nn.ReLU(),
            nn.Dropout(0.3),            # Regularization
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, num_classes)  # Output: predict demographic
        )

    def forward(self, x):
        return self.net(x)

# Use it
adversary = AdversaryNetwork(num_classes=3)  # For 3 races
optimizer = optim.Adam(adversary.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()

# Train on model predictions
X_adv = torch.FloatTensor(model_predictions.reshape(-1, 1))
y_race = torch.LongTensor(actual_races)

for epoch in range(300):
    logits = adversary(X_adv)
    loss = loss_fn(logits, y_race)
    loss.backward()
    optimizer.step()
```

Part 5: Explainability

SHAP

What: Feature importance, interpretability **Why:** Shows which features leak demographics
When to use: Understanding model decisions, feature attribution

Installation:

```bash
```

```
pip install shap==0.42.0
```

Key functions:

TreeExplainer (for tree-based models)

```python
import shap
from sklearn.ensemble import RandomForestClassifier

# Train model
model = RandomForestClassifier()
model.fit(X_train, y_train)

# Get SHAP values
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)  # Shape: (n_samples, n_features)

# Visualize
shap.summary_plot(shap_values, X_test, feature_names=feature_names)
shap.dependence_plot("feature_name", shap_values, X_test)
```

KernelExplainer (for any model)

```python
# For custom models (including adversary)
explainer = shap.KernelExplainer(
    model.predict,
    shap.sample(X_test, 100)  # Sample for faster computation
)
shap_values = explainer.shap_values(X_test)
```

Feature Importance Per Group

```python
```

```python
# Get SHAP values
shap_values = explainer.shap_values(X_test)

# Compare between groups
white_mask = demographics == 'White'
black_mask = demographics == 'Black'

white_importance = np.abs(shap_values[white_mask]).mean(axis=0)
black_importance = np.abs(shap_values[black_mask]).mean(axis=0)

gap = np.abs(white_importance - black_importance)

# Identify proxies
for feature, g in zip(feature_names, gap):
    if g > 0.05:  # Large gap = proxy
        print(f"PROXY: {feature} (gap: {g:.3f})")
```

Part 6: Data & Databases

SQLite

What: Lightweight database **Why:** Store audit history over time **When to use:** Tracking metrics monthly, drift detection

Installation:

```bash
# Built into Python, no install needed
```

Usage:

```python
```

```python
import sqlite3

# Connect
conn = sqlite3.connect('fairness_history.db')
cursor = conn.cursor()

# Create table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS audits (
        timestamp DATETIME,
        model_id TEXT,
        metric TEXT,
        value REAL
    )
''')

# Insert
cursor.execute(
    "INSERT INTO audits VALUES (?, ?, ?, ?)",
    ('2024-03-15', 'sepsis_v1', 'auc_gap_race', 0.25)
)
conn.commit()

# Query
cursor.execute(
    "SELECT * FROM audits WHERE metric = ?",
    ('auc_gap_race',)
)
results = cursor.fetchall()

conn.close()
```

Part 7: Visualization

Matplotlib

What: Basic plotting **Why:** Simple charts, histograms, trends **When to use:** Quick visualizations, static plots

Installation:

```bash
```

```bash
pip install matplotlib==3.7.0
```

Usage:

```python
import matplotlib.pyplot as plt

# Line plot (drift detection)
plt.plot(months, auc_values)
plt.xlabel('Month')
plt.ylabel('AUC')
plt.title('Algorithm Drift')
plt.savefig('drift.png')

# Histogram
plt.hist(predictions[demographics == 'White'], bins=30, alpha=0.5)
plt.hist(predictions[demographics == 'Black'], bins=30, alpha=0.5)
plt.savefig('distribution.png')
```

Plotly

What: Interactive plotting **Why:** Beautiful dashboards, hover details **When to use:** Interactive reports, hospital dashboards

Installation:

```bash
pip install plotly==5.13.0
```

Usage:

```python
```

```python
import plotly.express as px
import plotly.graph_objects as go

# Bar chart
fig = px.bar(
    x=['White', 'Black', 'Asian'],
    y=[0.87, 0.62, 0.75],
    title='AUC by Race',
    labels={'x': 'Race', 'y': 'AUC'}
)
fig.show()

# Line chart
fig = px.line(
    x=months,
    y=auc_gaps,
    title='Fairness Drift Over Time'
)
fig.show()
```

Streamlit

What: Web dashboard framework **Why:** Interactive reports for hospitals **When to use:** Displaying audit results

Installation:

```bash
pip install streamlit==1.20.0
```

Usage:

```python
```

```python
import streamlit as st
import json

st.set_page_config(page_title="FairnessOps", layout="wide")

# Load report
with open('audit_report.json') as f:
    report = json.load(f)

# Display
st.title("FairnessOps Audit Report")

col1, col2, col3 = st.columns(3)
col1.metric("AUC Gap", f"{report['auc_gap']:.3f}")
col2.metric("Leakage", f"{report['leakage']:.3f}")
col3.metric("Alert Level", report['alert_level'])

# Tabs
tab1, tab2, tab3 = st.tabs(["Overview", "Demographics", "Proxies"])

with tab1:
    st.write("Summary information")

with tab2:
    st.write("Demographic breakdown")

with tab3:
    st.write("Proxy bias findings")

# Run
# streamlit run app.py
```

Part 8: Development & Testing

Jupyter

What: Interactive notebooks **Why:** Explore data, test code iteratively **When to use:** Exploratory work, prototyping

Installation:

```bash
```

```
pip install jupyter==1.0.0
```

Usage:

```bash
jupyter notebook
# Opens http://localhost:8888
# Create .ipynb files, write code in cells, run with Shift+Enter
```

Pytest

What: Testing framework **Why:** Catch bugs early **When to use:** Unit testing your code

Installation:

```bash
pip install pytest==7.3.0
```

Usage:

```python
# tests/test_adversary.py
def test_adversary_trains():
    adversary = AdversaryNetwork()
    X = torch.randn(100, 1)
    y = torch.randint(0, 2, (100,))

    # Train
    optimizer = optim.Adam(adversary.parameters())
    for _ in range(10):
        out = adversary(X)
        loss = nn.CrossEntropyLoss()(out, y)
        loss.backward()
        optimizer.step()

    # Check it improved
    assert loss.item() < 0.5

# Run: pytest tests/test_adversary.py
```

Part 9: ML/DL Techniques You'll Actually Use

1. Neural Networks (Adversary)

What: Multi-layer neural network **Size:** Small (2-3 layers, 16-32 neurons) **For:** Detecting proxy bias

Architecture:

```
Input (1) → Linear(1, 32) → ReLU → Linear(32, 16) → ReLU → Output(2-5)
```

Why small?

- Simple task (predict demographic from single prediction)
- Avoid overfitting
- Fast training

---

2. Logistic Regression (Baseline)

What: Linear classifier **For:** Comparing against neural network **Why:** Simplest possible adversary

```python
from sklearn.linear_model import LogisticRegression

baseline = LogisticRegression()
baseline.fit(X, y)
baseline_accuracy = baseline.score(X_test, y_test)

# Compare to neural network
print(f"NN accuracy: {nn_accuracy}")
print(f"Baseline accuracy: {baseline_accuracy}")
# If NN not much better, maybe problem is linear
```

---

3. Cross-Entropy Loss

What: Loss function for classification **For:** Training adversary **Why:** Standard for multi-class problems

```python
loss_fn = nn.CrossEntropyLoss()

# For binary (Black vs White)
logits = model(X)  # Shape: (batch_size, 2)
loss = loss_fn(logits, y_true)  # y_true: shape (batch_size,)
```

---

4. Adam Optimizer

What: Optimization algorithm **For:** Training adversary **Why:** Works well, automatically tunes learning rate

```python
optimizer = optim.Adam(model.parameters(), lr=0.01)

for epoch in range(300):
    loss = compute_loss()
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

---

5. Dropout

What: Regularization technique **For:** Prevent overfitting **Why:** Randomly removes neurons during training

```python
class Adversary(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 32),
            nn.ReLU(),
            nn.Dropout(0.3),  # Drop 30% of neurons
            nn.Linear(32, 2)
        )
```

---

6. Bootstrap Resampling

What: Resampling technique **For:** Confidence intervals **Why:** No assumptions about distribution

```python
gaps = []
for i in range(500):
    idx = np.random.choice(len(data), len(data), replace=True)
    gap = compute_gap(data[idx])
    gaps.append(gap)

ci_lower = np.percentile(gaps, 2.5)
ci_upper = np.percentile(gaps, 97.5)
```

---

7. K-Fold Cross-Validation

What: Validate on different data splits **For:** Ensure adversary doesn't overfit **Why:** More robust evaluation

```python
from sklearn.model_selection import KFold

kf = KFold(n_splits=5)
for train_idx, val_idx in kf.split(X):
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    # Train on train_idx, evaluate on val_idx
    train_accuracy = train_and_evaluate(X_train, y_train, X_val, y_val)
```

---

8. Feature Scaling (Normalization)

What: Standardize feature ranges **For:** Helps neural networks train better **Why:** Large values cause gradient issues

```python
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

# For adversary (predictions are already 0-1, so maybe skip)
# But for clinical features, do this
```

Complete Tech Stack Summary

Required

```
Language:        Python 3.9+
Data:            NumPy, Pandas
ML Metrics:      Scikit-learn, SciPy
Neural Networks: PyTorch
Explainability:  SHAP
Database:        SQLite
Visualization:   Matplotlib, Plotly, Streamlit
Testing:         Pytest
Notebooks:       Jupyter
```

Installation One-Liner

```bash
pip install numpy pandas scikit-learn scipy torch shap matplotlib plotly streamlit j
```

Requirements.txt

```
numpy==1.24.0
pandas==2.0.0
scikit-learn==1.2.0
scipy==1.10.0
torch==2.0.0
shap==0.42.0
matplotlib==3.7.0
plotly==5.13.0
```

```
streamlit==1.20.0
jupyter==1.0.0
pytest==7.3.0
```

ML/DL You WON'T Need (For Now)

```
❌ Transformers/BERT (for NLP tasks)
❌ Convolutional Networks (for images)
❌ Recurrent Networks (for sequences)
❌ Reinforcement Learning
❌ Generative Models (GANs, VAE)
❌ Graph Neural Networks
❌ Ensemble methods (Random Forests, XGBoost)
   (You COULD use for baseline model, but not required)
```

ML/DL Pipeline For FairnessOps

```
1. Load Data
   ├─ pandas.read_csv()
   ├─ numpy preprocessing
   └─ scikit-learn train/test split

2. Train Hospital Model (or use existing)
   ├─ RandomForestClassifier OR
   ├─ LogisticRegression OR
   └─ Your own model

3. Get Predictions
   └─ model.predict_proba(X_test)

4. Train Adversary
   ├─ Create PyTorch network
   ├─ Adam optimizer
   ├─ CrossEntropyLoss
   └─ Train 300 epochs

5. Evaluate Adversary
   ├─ Compute accuracy
   ├─ Compare to baseline
```

```
   └─ Measure leakage

6. Get SHAP Values
   ├─ shap.TreeExplainer() or KernelExplainer()
   ├─ Compute feature importance per group
   └─ Identify proxies

7. Bootstrap CI
   ├─ numpy.random.choice()
   ├─ Resample 500 times
   └─ Get percentiles

8. Visualize
   ├─ matplotlib for static plots
   ├─ plotly for interactive
   └─ streamlit for dashboard

9. Store Results
   ├─ json.dump() to report
   ├─ sqlite for history
   └─ csv for logs
```

What To Learn First (Priority Order)

1. **NumPy + Pandas** (1-2 hours)
   - Load data, filter, group, compute
2. **Scikit-learn** (1 hour)
   - Metrics (AUC, confusion matrix)
   - Train/test split
3. **PyTorch basics** (2-3 hours)
   - Tensors
   - Simple network
   - Training loop
4. **Your adversary** (1 hour)
   - Build small network
   - Train on predictions
   - Measure accuracy

5. **SHAP** (1 hour)
    - Get feature importance
    - Compare between groups
6. **Streamlit** (1-2 hours)
    - Display results
    - Interactive dashboard

Total: ~8-10 hours to understand everything

Generated: FairnessOps Tech Stack & ML/DL Techniques
Version: 1.0